

System	Function	Limitation
Firewall	Filters IP/port headers	Cannot inspect payloads
IDS	Detects and logs suspicious packets (header+payload)	logs/alert not block
IPS	Detects and blocks packets (header+payload)	Can drop legitimate traffic

TCP/IP's design limits firewall accuracy (fragmentation, spoofed src/dst).

DNS attacks:

- Application-Layer protocol
- Flow การทำงานทั่วไป
 - ผู้ใช้งานขอชื่อโดเมน ผู้ให้บริการ URL เช่น www.chula.ac.th ในเบราว์เซอร์ → เครื่องจะต้องหาว่าโดเมนนี้มี IP Address อะไร
 - ระบบปฏิบัติการหรือเบราว์เซอร์จะขอ DNS cache ในเครื่อง (local) ก่อน
 - ถ้ามีข้อมูล (ถ้าไม่หมดอายุ TTL) จะใช้ IP นั้นทันที
 - else -> ถาม DNS Resolver (ของ InternetServiceProvider หรือองค์กร)
 - Resolver ตรวจสอบลำดับขั้นของ DNS
 - ตรวจสอบใน cache
 - ถ้ายังไม่รู้จักถาม Resolver จะทำการค้นแบบ "recursive"
 - ถาม Root DNS Server → ได้ชื่อของ Top-Level Domain (TLD) เช่น .th
 - ถาม TLD Server (.th) → ได้ชื่อของ Authoritative Server ที่ดูแล chula.ac.th
 - ถาม Authoritative Server (chula.ac.th) → ได้ IP ของ www.chula.ac.th
 - Resolver ส่งผลกลับให้เครื่องผู้ใช้
 - Resolver จะเก็บผลไว้ใน cache เพื่อใช้ในอนาคต
 - ผู้ใช้งานรับ IP เช่น 161.200.192.4
 - นำ IP ที่ได้ไปเปิดการเชื่อมต่อผ่าน TCP หรือ HTTPS
- ปัญหา:
 - รับมี part additional ที่ใส่ <name>ip> ตอน authoritative ส่งคืนให้ DNS resolver
 - DNS resolver เชื่อสิ่งที่ DNS Server ส่งกลับมา 100%
- Spoofing (Kaminsky): DNS Spoofing -> เป็นวิธีการ
 - ตอน DNS resolver ถาม DNS Server -> ทำไฟล์ Cache Poisoning
 - เมื่อก่อนใน Response, port ส่วนใหญ่ที่เป็น 53 ดังนั้นเดาแต่ queryId ก็พอ ซึ่งแค่ 16 bits เท่านั้น เหมือนอิงตาม Birthday paradox โอกาส Q=256.R=256 แล้วถูกถึง 63 %
 - พยายามหาทำ DNS resolve ของเครื่องเป็นค่าสว่ใจ 2^16 256 อัน (Q = 256)

$$P_{fail} = \frac{65,536 - 256}{65,536}$$

- เราปลอมไป 256 อัน (R = 256), lower_bound ต้องให้ R มีค่า

$$P_{256fail} = \left(\frac{65,536 - 256}{65,536}\right)^{256} = 0.367$$

$$P_{success} == 1 - 0.367 = 0.63$$

Defense:

- Source QueryId Random (16 bits -> 32 bits) -> แตกยากเพราะต้องแก้ Protocol
- increase-port randomization: เปลี่ยนจาก fixed ทำให้ออกมาเพิ่มเป็น 2^32 bits
- Cache poisoning
 - ผู้ใช้งานส่ง DNS response ปลอม ที่มี Additional Record แฉกโดเมนอื่นเข้ามาด้วย (ส่งด้วย DNS Spoofing)
 - ตัวอย่าง: ผู้ใช้ถาม IP ของ a.com แต่ผู้โจมตีแบบปลอม b.com = 6.6.6.6 มาด้วย ซึ่งจริงๆ b.com = 1.1.1.1, 6.6.6.6 เป็น ปลอมทั้งคู่
 - Defense:** แก้ด้วยไม่แค่นั้น แต่มันก็จะทำให้อ Perf แย่ จึงเปลี่ยนเป็นแคชได้เฉพาะ web ที่อยู่ใน domain เดียวกัน (Bailiwick Checking) เช่น ns.a.com, mail.a.com

- Rebinding: เปลี่ยน IP ที่ผูกกับชื่อโดเมนให้เป็น Internal ip, TTL อันแรกน้อย รอบ 2 ส่งให้เป็น Internal IP แล้วให้ JS ที่ส่งไปจับข้อมูล

- DNS Spoofing ไม่ใช่ Required Condition: ถูกต้องครบ (ปกติแต่หลุดก็เกิดลิงก์ก็พอแล้ว)
- แต่ DNS Spoofing เป็น Catalyst (ตัวเร่ง): ช่วยบังคับให้คนที่ไม่ยอมกดลิงก์ก็แปลกๆ กลายเป็นเหยื่อของ DNS Rebinding ได้
- Defense:**
 - ส่งเบราว์เซอร์: บังคับป้องกัน DNS rebinding, refuse mid-session IP switch
 - ส่งเครื่องช่วย/Resolver: บังคับการ resolve ชื่อสาธารณะให้เป็น private IP ranges
 - Firewall: ป้องกันการเข้าถึงพอร์ตจัดการจากเครือข่ายภายนอก

- DNSSEC (auth & integrity) -> เพิ่ม "ลายเซ็นดิจิทัล" (Digital Signature) -> แก้ DNS Spoofing -> แก้ Cache Poisoning

TCP Vulnerabilities

- Spoofing:** Guess sequence number (1/2³¹ chance). ผู้โจมตีต้องเก็บที่เกิดปลอมที่แสดงตัวเป็นเครื่องหนึ่งของการเชื่อมต่อ เพื่อแทรกข้อมูลหรือ hijack stream.
- Reset attack:** Inject TCP RST to terminate connection. ส่งแพ็กเก็ตที่มีเลข RST เพื่อสั่งให้เครื่องส่งบิตการเชื่อมต่อทันที. ต้อง 4-tuple (srcIP, srcPort, dstIP, dstPort) และ sequence number ที่อยู่ใน window
- SYN Flood:** ส่ง SYN จำนวนมาก (มัก spoof source IP) ทำให้เซิร์ฟเวอร์สร้าง state (half-open) ไร่ ACK ที่จะไม่มา -> ีคิว backlog เต็ม ไม่รับการเชื่อมต่อใหม่จากผู้ใช้งาน
 - Good fixes:** SYN cookies, ไม่ใช่ backlog แต่ encrypt แล้วใช้คีย์มาส่งกลับตัวรับ.
 - With SYN cookies, server does not store state. Instead, it encodes connection info (client IP, port, timestamp, etc) inside the SYN-ACK's sequence number.
 - If the client is real, it replies with an ACK that includes this value.
 - Bad fixes:** Increasing queue or shortening timeout.

Asymmetric Encryption

- Keep **private key** secret; share **public key** freely.
- One key pair per person -> good scalability.
- Use cases:
 - Confidentiality** (encrypt with public key) -> only private key can read
 - Integrity** (verify with public key) -> digital signature
 - digital signature = hash(received message) ต้องทำกับ public(DigSig)

3. Missing Link – Authentication

Even with public keys, we cannot prove ownership without binding a key to an identity. -> ไม่รู้ว่าเป็นของใครเพราะมันคือ public key ไม่ใช่ identity signature -> ทำให้ public key นั้นเชื่อถือผ่านคนกลางที่เข้าเชื่อถือ (Certificate Authority)

Digital Certificate

- Binds an entity's **public key** to its identity attributes.
- Issued and digitally signed by a trusted third party (CA).
- A ส่ง publicKeyA ให้ CA แล้ว CA ให้ Digital Certificate ให้ซึ่งมี privateKeyC(Cert,publicKeyA)
- B ที่ถือ CA หรือคนที่ถือ publicKeyC A ส่ง Digital Certificate ให้ B decrypt ก็จะเห็น publicKeyA และ Cert ที่ CA ออกให้ A -> น่าเชื่อถือ

5. Trust Model – Chain of Trust

If a trusted entity signs a certificate, you can trust the certificate and its subject.

Example chain:

Digicert Inc (Root CA) – Thawte TLS RSA (CA) – www.chula.ac.th

6. Anatomy of a Certificate

- Issuer: ใครออกให้
- Subject: เจ้าของใบรับรอง
- Subject Public Key: PublicKey
- Issuer Digital Signature: ลายเซ็น ที่ Issuer ใช้ private key เช่น certificate ของ Subject -> ใช้ตรวจว่าใครเป็นผู้ Issue (chain ได้มาจาก root server)

PKI Lifecycle

- Enrollment: ลงทะเบียนขอใบรับรอง
- Issuance: ออกใบรับรอง
- Validation: ตรวจสอบความถูกต้อง

1. Encryptio

Type	Strengths	Weaknesses
Hash/Digest	Fastest, ensures integrity	No confidentiality
Symmetric Encryption	Fast, provides confidentiality	Poor scalability (n เลือก 2 keys needed), integrity uncertain, no non-repudiation

Asymmetric Encryption	Confidentiality, integrity, scalable (2n keys needed), non-repudiation	100–1000× slower than symmetric encryption
-----------------------	--	--

Combination of these methods can solve most issues **except authentication**.

Anyone can create and sign their own certificate. Trust depends on manual verification by the recipient.

Self-Signed Cert

Root CA เป็นของตัวเอง

8. Legal and Operational Issues

เชื่อมโยง Infrastructure เพราะมีการนำ real-world มาช่วยกำกับ ไม่ใช่แค่ใช้คอมพิวเตอร์ digital world

PKI Parties and Roles

- CA (Certificate Authority):** key generation, issuance, revocation, backup, cross-certification: ทำ Cert ให้
- RA (Registration Authority):** verify identity (face-to-face or remote), revoke requests: ตรวจสอบขอเพิ่มหรือลบขอส่งให้ CA ทำ
- VA (Verification Authority):** ตรวจสอบ Cert ก่อนใช้งาน
- Certificate Distribution System:** certificates + CRLs (typically via LDAP):
 - ถ้ามีการดาวน์โหลดตรวจสอบสถานะใบรับรอง ในระบบ PKI, มีใบมีการดาวน์โหลดตรวจสอบใบรับรอง ของผู้ให้บริการอื่นไว้เพื่อ
 - เผยแพร่ CRL เพื่อให้อีกทุกคนรู้ว่าใบรับรองใดถูกยกเลิกแล้ว
- Other components:** VA, policy database, PKI-enabled apps

Buffer overflow occurs when data exceeds buffer limits and overwrites **adjacent memory**, potentially altering control flow and executing malicious code. **Secure Bit** is a hardware-based mechanism designed to prevent such attacks by tracking whether input data is trusted.

3. Buffer Overflow Works

Example in C using `gets()` -> overwrites stack memory. 'l' = 0x31 = 49 **Attack types:**

- Return address modification (stack smashing) = เราเขียนจนเกิน Return Address ทำให้มันไปรันอะไรก็ได้. เราเอาโค้ดมาแก้ก่อนให้มัน Return กลับมารันยังได้
 - Mandatory Condition
 - 1. Input แปลกเขียนได้
 - 2. Address สำคัญ (Return address) โดเมน
 - Sol: ไม่ให้เกิด 2 conditions พร้อมกัน
 - Input Range/Value Validation : ของทำให้เราเขียนได้เท่านั้น
 - Critical Code Validation : ดูว่าถูกแก้ไขไหม -> digital signature

Static Analysis (Lexical/Semantic Analysis)

- Lexical (ดูคำศัพท์): ตรวจสอบจาก code อย่างค่าแบบ strcpy มีโอกาสสูงทำให้เกิด buffer overflow
 - Tools: ITSA(String Matching), FlawFinder, RATS
- Semantic (ดูความหมาย/บริบท): ตรวจสอบ "ตรรกะ" ถูกประกาศตัวแปรไว้กี่ครั้งแล้ว แต่ต้นเอาข้อความไปใส่ -> "ผิดประเภท"
 - Tools: Splint, BOON
- Pros: Detects known issues pre-deployment; ค้นหาคำผิด
- Cons: No runtime protection (ไม่ช่วยตอนรันแล้ว), false positives (Warning เยอะเกินไป/ผิด)

4.3 Isolation

- Non-executable memory (NX), sandboxing
 - Memory บางส่วนไม่อนุญาตให้ execute
 - ไม่ใช่ root cause เพราะ NX ก็มีการขัดโค้ดในรันได้ แต่ยังไม่ใช้คำสั่งที่มีอยู่แล้วในเครื่องได้ ขัดติดขัดต่อกันจนกลายเป็นคำสั่งร้ายแรง เรียกว่า ROP (Return-Oriented Programming)

4.4 Secure Boost

- hash กับ kernel code ไว้เป็น digital signature -> ถ้าไม่ตรงไม่ boost

5. Theory

Definition: Buffer overflow happens when external input modifies addresses. **Theorem:** If an address cannot be modified (or modification can be detected), buffer-overflow attack is impossible. **Corollary:** Preserving address integrity prevents buffer overflow.

7. Secure Bit Concept

Idea: Tag all data from untrusted domains with a "Secure Bit."

- When data crosses domain boundaries, the bit is set.
- CPU disallows such data as jump/control targets.
- Ensures "untrusted input ≠ executable control."

File Carving

- Reassemble deleted or fragmented files from disk space.
- Computer don't immediately remove data that is deleted.
 - After delete the original data is still present, but marked as unallocated space, even some or all of the data has been overwritten, the remaining data can still be carved and reviewed.

Threat	Mitigation
Spoofing (ปลอมตัว)	Authentication
Tampering (แก้ไข/ปลอมแปลง)	Integrity
Repudiation (ปฏิเสธการรับผิดชอบ)	Non-repudiation
Information Disclosure (เปิดเผยข้อมูล)	Confidentiality
Denial of Service (ไม่ทำงาน)	Availability
Elevation of Privilege (เพิ่มสิทธิ์)	Authorization
Disable unused ports/services.	
Turn off UDP (if your service does not need)	
Restrict network ranges. (only allow for trusted network)	
Authenticating connections.	
Harden ACL (Access Control Lists). (Admin = full access, Everyone = read only, Service = Read&Write)	
Run services with least privilege.	

Secure Design Principles

- Plan security from the start, not after implementation.
- Attack Surface Reduction (ASR):**
 - Attack Surface: Where attacker can enter or extract data from a system
 - Less code running = fewer vulnerabilities
 - Defense in Depth – multiple layers of protection -> remove single point of failure
 - Least Privilege – limit permissions
 - Secure Defaults – turn off unnecessary features -> less stuff to attack
- Each user/role can do what to this resource, attached to Resource
- ของ (resource) ที่เพิ่มหรือลดและแบ่งว่าใครทำอะไรกับของนี้ได้บ้าง
- Capability-based System** -- subject-centric, like tokens/keys
 - This user/role can do what to each resource, attached to Subject
 - บุคคล (user/role) ที่เพิ่มหรือลดและว่าทำอะไรกับของนี้ได้บ้าง

Security Models

- define Policy (Blueprint) -> Guideline
- Multilevel Security: Within Org, MAC
 - Bell-LaPadula (Confidentiality): Focus = Secrets
 - No Read Up (NRU), No Write Down (NWD)
 - Prevents information leaks.
 - หน่วยงานราชการ, กองทัพ ที่ "ความลับ" สำคัญที่สุด
 - Biba (Integrity): Focus = Trust/Truth
 - No Read Down, No Write Up
 - Prevents corruption from lower-integrity sources.
 - ธนาคาร, ระบบบัญชี ที่ "การขาดค่า" สำคัญที่สุด
- Multilateral Security: China Wall Model
 - Conflict of Interest: Dynamically blocking access to competitor data.
 - สำนักงานโฆษณา, บริษัทโฆษณา, ตลาดหลักทรัพย์ เพื่อป้องกันข้อมูลภายในของตลาด
 - เอาข้อมูลของลูกค้า A ไปปนกับลูกค้า B

Address Protection

- Canary Words: มาจาก Canary Bird (นกขมิ้น->อ่อนแอ ตายง่าย) -> จะถูกแทนที่บน RET โจนเนอร์
 - กั้นระหว่าง Buffer กับของสำคัญๆ ถ้า overflow Canary ต้องถูกแก้
 - ถ้ามีค่าเป็น "Null Byte" (0x00) อยู่ด้วย เพราะ **strcpy** หยุดทำงานเมื่อเจอ Null Byte
 - ไม่ใช่แค่ใส่สัญญาณเตือน แต่ยังเป็น "กำแพง" ที่ทำให้ **strcpy** หยุดทำงานด้วย
 - StackGuard:** Canary word before return address -> Detect Overwrite
 - ปัญหา: ถ้า hacker f Canary Word ก็แก้ไขในโค้ดก็ได้
- Address Encode: Encrypt Address ทำให้ hacker แก้ได้ แต่เนื่องจากไม่รู้ว่า Encrypt อย่างไร เราย Decrypt ก็จะไม่รู้ที่ไหน **ผู้ใช้** ซึ่งมีโอกาสสูงจะ Crash -> ทำให้เราไม่ไปรันโค้ดที่ hacker เอาไว้ให้
- PointGuard:** Encrypts pointers. Protects function pointers
- SPEF:** Encrypted instruction/code
- Instruction Set Randomization (ISR):** เข้ารหัส/ชุดคำสั่งด้วย per-process key (เช่น XOR) ทำให้โค้ดที่โจมตีเขียนมาไม่ตรงกับ ISA ที่ถูกถอดรหัสจริง (Columbia / Drexel)
 - ปกติ ADD อาจจะเป็น Opcode 0x01 ในทุกที่เจอ.
 - ISR จะสุ่มให้ Process A ใช้ ADD = 0x55, Process B ใช้ ADD = 0xAA, ซึ่ง hacker เค้าไม่รู้ได้
- ปัญหา: Performance, Compatibility
- Copy of Address: เก็บ return addr สองที่จริง, เวลาจะ ret จะเทียบสองค่าถ้าไม่ตรงก็ถือว่าเขียนทับ
 - Split Stack:** Separates control(ret)/data(buffer) stack, มี ret ที่เดียว แยกไว้อยู่
 - ปัญหา: เทียบสองที่ที่โดนอยู่
- Tags: บิดพิเศษ meta data ที่ติดไปกับแต่ละคำของหน่วยความจำ (memory word) เพื่อบอกว่าให้แสดงว่า "ข้อมูลนี้มาจากที่มาใด" ถ้าจะใช้ทำนี้เป็น Addr แดต้นมาจาก input -> "ไม่อนุญาต"
- Input Protection:** Prevents untrusted data as control data -> Secure bit

Bounds Checking: Validates memory access: ของทำให้เราเขียนเท่านั้น — ทางคนที่ถูกส่งแต่แพง

- Array Bounds Checking: Software-level checks
- Segmentation / hardware bounds: Hardware-level checks
- LibSafe / LibVerify:** พ่อแม่แทน libc เพื่อตรวจสอบ bounds และการจัดการ I/O/strcpy กัน
- เขียนจริง (Bell Labs)
- ปัญหา: ข้างล่างช้ามาก (ตัวอย่างในเอกสารอ้างถึง ≈30x slowdown ในบางกรณี)

Obfuscation: ASLR (Address Space Layout Randomization) -> เค้าไม่รู้ว่า RET อยู่ไหน, Hack ได้แต่ยากขึ้นเฉยๆ จากเมื่อก่อนแค่อ่าน Source Code ก็ทำได้แล้ว

4. Data Hiding Techniques : Disguises information from normal view

- Rename:** Changing name and extension -> make it look harmless/irrelevant
 - Weakness: Forensic tool check file header can compare the extension
- Attributes hiding:** Mark as Hidden in OS -> disappears from normal file explorer view
 - Weakness: Visible if system show hidden files
- Bit-shifting:** Shift binary data to hide content -> file become unreadable (lightweight encryption) -> but easy to crack.
- Encrypt/Password:** Data unreadable without key
- Partition hiding (ไม่ mount): diskpart remove letter** -> OS won't display that partition (OS wont mount), though data still exists.
 - Weakness: Visible to forensic tools that scan unallocated space, analyze disk space
- Marking bad clusters (Mount เค้าไม่ให้เพราะถูก mark ว่าพัง):** FAT file system, attacker can manually mark good clusters as "bad", so the OS skips them, assuming they are damaged.
 - Weakness: Detectable with modern forensic tools but invisible to casual users.

Steganography & Steganalysis

- Steganography:** hiding information inside other files (images, audio).
 - เอาข้อความลับซ่อนไว้ในรูปภาพ โดยเปลี่ยนค่า บิตเล็ก ๆ (LSB – Least Significant Bit) ของพิกเซล
 - ส่งข้อความในไฟล์เสียง/วีดีโอโดยเปลี่ยนค่าที่มนุษย์ฟังไม่ต่าง
 - Keys: Human not noticable to these changes
- Steganalysis:** detecting and analyzing Steganography.
- Digital watermarking:** hides ownership info -> บอกว่าไฟล์นี้ของใคร, พยายามเอาข้อมูล. Steganography that for license purpose not hiding information เช่น ข้างภาพซ่อนชื่อใน .jpg
- Authorization:** "What can you do?" -> **Policy will tell you**
 - Policy (Law):** Help to decide
 - App-specific decision-making based on policy : Something action is different in different application -> No general AuthZ can use for every application
 - Type Enforcement (Police): Enforce to follow policy

- Security Policy:** Clearly define what is secure or insecure. We cannot say something is "safe" unless we define what "safe/unsafe" looks like
- Secure System:** Start Secure + No Insecure Moves = Always Secure.
 - Starts in a secure state
 - Cannot enter an insecure state.

Access Control Models

- Mandatory Access Control (MAC)
 - Admin-defined policies; users untrusted.
 - Multilevel system : Higher Level is king.
- Discretionary Access Control (DAC)
 - Owner-defined policies
 - Common in personal computing/drive.
- Role-Based Access Control (RBAC)
 - Who know about role defined policies
 - Based on least privilege; access tied to job roles.
 - No clear definition on how to define a role lead to **Role Explosion**
 - "Nurse" role.
 - "Nurse who works night shift" role
 - "Nurse who works night shift and handles X-rays."
 - Suddenly, you have 1,000 roles, and it's a mess.
- Attribute-Based Access Control (ABAC)
 - Decisions use attributes to map access ability.
 - Attributes : User attr, Resource attr, Environment attr
- Modern Access Control
 - OAuth 2.0 / OIDC:** Limit-Accessed Pass
 - Zero Trust Architecture:** Never Trust, Always Verify

Policy Development Steps

- Analyze security model (Bell-LaPadula, Biba, China Wall).
- Choose access control model (MAC, DAC, RBAC, ABAC).
- Convert policy – Access Control Matrix.
- Implement via Type Enforcement (ACLs, Capabilities).